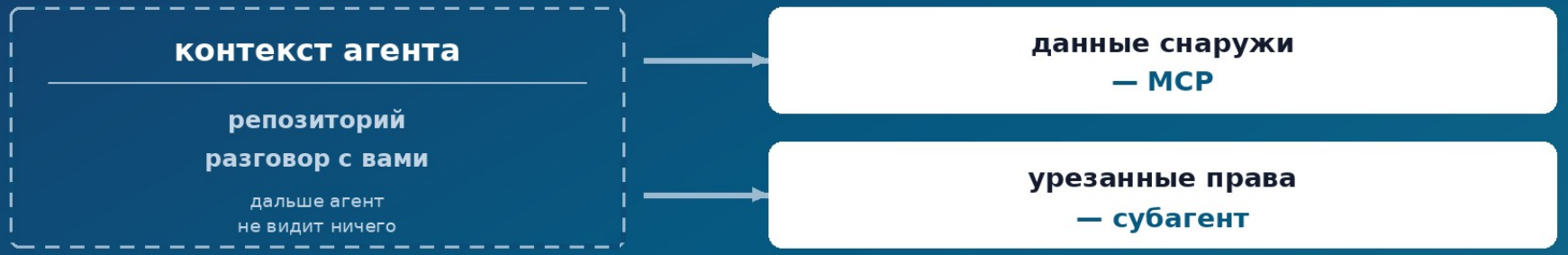
Доступ наружу и урезанные права

- Две недели назад агент знал про этот проект всё. В понедельник он уверенно ответил мимо.

«Агент не поглупел — так что же перестало помещаться?»



хук	скилл	MCP	субагент
закрыты в прошлый раз	закрыты в прошлый раз	решаем сегодня	решаем сегодня

Где мы остановились

СТУПЕНЬ	ПУТЬ ИЛИ АРТЕФАКТ	СТАТУС
Хук	`.claude/settings.json` + `.claude/hooks/selftest-branch-guard.sh`	коммит есть
Скилл	`.claude/skills/deploy/SKILL.md`	только план, коммита нет
CLAUDE.md	было 24 строки / 4 раздела → стало 40 строк / 5 разделов	коммит есть
DECISIONS.md	документационный слой репозитория	без изменений

«Четыре вещи, и все четыре — внутри репозитория. Пока работа была здесь, агент держал проект целиком.»

Две строки закрыты — обе про то, что внутри

СТУПЕНЬ	ПРЕДОТВРАЩАЕМ / РЕШАЕМ	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
Хук	Решаем — правило уже нарушено	Отклоняет один паттерн записи в `main` из шести способов коммита	На остальных пяти молчит; человека с терминалом не видит вовсе
Скилл	Решаем — по сигналу перегрузки файла инструкций	Загружает файл по точному совпадению описания-триггера	Без точного триггера не сработает вообще — содержимого файла до срабатывания в контексте нет
MCP			
Субагент			

Хук и скилл работают внутри репозитория. Обе сегодняшние строки — про его край.

Два промаха, причины противоположные

КОГДА И ЧТО	ПОЧЕМУ ТАК ВЫШЛО
Понедельник. Ответил по репозиторию — двух просьб заказчика в ответе не оказалось. Сорванный срок.	Внутри их нет вовсе. Контекст хоть вдвое больше — там они всё равно не появятся.
Среда, рабочий продукт. Прочитал всё, где правило могло проверяться, собрал верный список — и начал переспрашивать то, что объясняли в начале сессии. Полчаса.	Всё нужное внутрь влезло — и вытеснило то, ради чего сессию открывали.

«Работа вышла за контекст агента — что вы заводите первым: приносите внутрь то, чего внутри нет, или выносите наружу то, что внутрь не помещается?»

Два блока — два ответа на один вопрос

БЛОК	ПЕРВЫЙ	ВТОРОЙ	ТРЕТИЙ
Принести внутрь то, чего внутри нет — MCP, доступ наружу	Агенту регулярно нужны данные, которых нет в репозитории. Решаем — и проверяем, что агент через сервер правда ходит.	Подключённые серверы занимают контекст до начала работы. Решаем.	—
Вынести наружу то, что внутрь не помещается — субагент, урезанные права	Чтение, нужное для одной правки, вытесняет из контекста саму правку. Решаем.	Задача пришла одной фразой, а сроку до неё два дня. Решаем.	Исполнителей несколько, рабочая папка у них одна. Решаем.

«Один и тот же предел. Два хода навстречу друг другу.»

1

«Я же дописал это в задаче»

Задача живёт в трекере и меняется без нас. У агента в чате — снимок, снятый руками сегодня утром.

MCP

Субагент

Файлы внутри, файлы снаружи — а трекер задач?

ЧЕМ АГЕНТ УЖЕ ПОЛЬЗУЕТСЯ

Файлы репозитория — агент читает их напрямую, это материал прошлых занятий.

Файлы, которые дали явно: вставили текст в чат, приложили документ.

Оба источника — файлы. Прочитал и работай.

А СПИСОК ЗАДАЧ В ТРЕКЕРЕ?

Заказчик ведёт его отдельно от проекта: в дереве репозитория такого файла нет.

Дать один раз и забыть не выйдет — список меняется без нас каждый день.

Снимок на сегодняшнее утро устареет к обеду. Нужен способ спросить систему в момент работы.

Такое обращение к системе называется вызовом API. Обычная догадка на этом месте: за него придётся платить написанной интеграцией. Проверим на пяти способах.

Сервер — у вас или снаружи, разговор один и тот же



КТО С КЕМ ГОВОРИТ

Агент говорит только с сервером, одним и тем же способом в обоих вариантах. С внешней системой говорит сервер: ключ к ней лежит у него.

ЧТО ПРИХОДИТ ОБРАТНО

Агенту возвращается ответ системы текстом — ровно то, что отдал сервер. Самой системы агент не касается.

Один сервер может ходить сразу в несколько систем — в каждую её способом: REST, gRPC, GraphQL, SQL.
Агент этих способов не знает: у него к любому серверу один разговор — MCP.

MCP и сервер, одним экраном

1 ЧТО ТАКОЕ СЕРВЕР

Отдельная программа, не часть Claude Code. Отвечает на запросы по протоколу.

stdio — локально процесс запускает Claude Code у вас на машине

http — удалённо сервис, который хостит и обновляет кто-то другой

Команда в конфиге исполняется буквально при старте сессии: что записано, то и запустится.

2 ГДЕ ОБЪЯВЛЯЕТСЯ

```
{ "mcpServers": { "github": {
      "command": "npx", … } } }
```

`.mcp.json` — в репозитории — область project

`~/.claude.json` — области local (дефолт) и user

Какая область и почему — разбираем дальше в кейсе

3 ЧТО ПРОИСХОДИТ ПРИ ПОДКЛЮЧЕНИИ

Транспорт поднялся — Claude Code САМА, без модели, спрашивает у сервера его список:

`tools/list` `prompts/list` `resources/list`

По умолчанию в контекст из этого ответа ложатся только имена инструментов и инструкция сервера о себе; полные схемы приходят позже, по требованию. Отсрочку отменяют флаг у отдельного сервера и переменная окружения — тогда все схемы грузятся при старте.

4 ВЫЗОВ: ПАРАМЕТРЫ АГЕНТ СОБИРАЕТ САМ

```
mcp__github__create_issue
{ title, body, … }
```

Агент сам поднимает нужный сервер и вызывает его — так же, как вызвал бы функцию: полное имя с префиксом сервера и аргументы, которые собрала модель.

5 КОНТЕКСТ И ЦЕНА — ТРИ СТАТЬИ

старт сессии имена инструментов и инструкция сервера — в контексте всегда

по умолчанию полную схему инструмента модель подтягивает по требованию

пришёл ответ результат вызова — отдельный бюджет: предупреждение за 10 000 токенов, обрезка за 25 000

Постоянная плата идёт за то, что сервер подключён. Цена вызова — отдельно, за каждый ответ.

6 ЧЕМ ОТВЕЧАЕТ СЕРВЕР

Результат приходит отдельным каналом от обнаружения возможностей — не тем же, что при подключении.

```
tool call
mcp__github__create_issue
```

Вызов подписан именем сервера — видно, что ответ пришёл именно оттуда, не из общих знаний модели.

Вторник, в который перенос руками дал сбой

Сайт живёт третий месяц. Заказчик уже месяц ведёт задачи в трекере — разработчик сам это предложил, чтобы правки не терялись в мессенджере. Перенос между трекером и чатом агента разработчик делает руками, несколько раз в неделю.

1. Утро вторника: открыл задачу «форма не отправляется с телефона», скопировал текст в чат агенту, ушёл делать правку.
2. Днём заказчик дописал в ту же задачу строку: «и с планшета тоже».
3. Разработчик её не увидел — в чате у него лежал утренний снимок. Вечером правка уехала заказчику.
4. Утром в задаче: «а планшет?». Час на вторую правку и письмо с объяснением, почему требование из задачи до работы не доехало.

Потерянный час — меньшая часть цены. Большая в том, что заказчик теперь перепроверяет, дошло ли до работы написанное им. Трекер живёт в отдельной системе и меняется каждый день без разработчика: снимок, снятый утром, к вечеру уже неточен.

Как дать агенту доступ к трекеру задач?

У кого из вас агент уже ходил наружу — в трекер, в базу, в документацию? Чем вы тогда дали ему доступ?

Выберите свой способ из пяти — и скажите, выдержит ли он регулярный доступ или вы бы сейчас взяли соседнюю карточку.

- **вызвать API напрямую**
- **подключить чужой сервер — удалённо**
- **подключить чужой сервер — локально**
- **написать свой сервер**
- **вызывать готовую команду gh каждый раз**

Кода интеграции не пишет ни один из пяти

СПОСОБ	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
вызвать API напрямую	агент сам пишет и шлёт запросы к эндпойнтам, сам разбирает ответ — отдельный код интеграции писать не нужно	заранее не знает спецификацию сервиса: какие есть точки, какие у них параметры и ограничения — без документации или swagger-описания агент их не угадает
подключить чужой сервер — удалённо	сервис, который кто-то хостит и обновляет сам; у вас не крутится отдельный процесс	управляете не вы: не видно, что сервис делает внутри и когда в нём что-то меняется; плюс обычные вопросы безопасности удалённого сервиса — кому доверяете ключ, у кого ещё есть доступ
подключить чужой сервер — локально	пакет исполняется у вас, конфигурация — строка в репозитории, видна всей команде в ревью; контроля больше, чем у удалённого	это всё равно живой процесс на вашей машине и чужой код — со всеми его зависимостями и правом на обновление
написать свой сервер	даёт ровно нужный набор операций и прав, без чужого кода в процессе	написать минимальный сервер сейчас быстро — для этого есть готовый плагин; дорого обходится поддержка: протокол, чужие пользователи, безопасность транспорта — отдельный проект рядом с основным
вызывать команду `gh` каждый раз	готовый официальный CLI GitHub — авторизацию и разбор ответа он уже решил за вас, кода писать не нужно	каждый вызов — отдельная команда без общего списка возможностей: агент не видит заранее, что вообще можно сделать, нужный вызов приходится знать заранее

Поворот. Интеграцию не пишут ни в одном из пяти способов: агент сам шлёт запрос и сам разбирает ответ. Разделяет пятёрку другое — знает ли агент заранее, что у системы можно спросить.

Что это оставляет. Свой сервер избыточен: официальный для трекера уже есть. `gh` и прямой вызов API закрывают вызов, известный заранее; растущий список операций они перекладывают на память разработчика.

Два законных варианта. Между «удалённо» и «локально» готового ответа нет: свежий официальный пример подключается удалённо, решение занятия — локальный процесс, потому что конфигурация становится кодом в репозитории, видимым в ревью.

Решаем: какие операции разрешить, потом — какие права

Ход 1 — что конкретно агент должен делать?

- читать открытые issue
- писать комментарий
- создавать черновик правки

Ход 2 — что даёт широкий персональный токен по умолчанию, и что из этого нужно?

ПРАВО ПО УМОЛЧАНИЮ	НУЖНО ДЛЯ НАШЕГО СПИСКА ОПЕРАЦИЙ?
все репозитории пользователя, включая приватные	нет — вычеркнуть
чтение и запись	частично — чтение нужно, запись (комментарий, черновик) нужна
управление настройками	нет — вычеркнуть
удаление	нет — вычеркнуть

Остаётся: один репозиторий, чтение issue, создание черновика правки.

Решаем: где хранить ключ и в какую область положить файл

Ключ — в переменную окружения. Файл конфигурации — в одну из трёх областей

Ход 3 — где хранить ключ? переменная окружения, не значение в файле

Ход 4 — куда положить сам файл?

local — **только у меня на этой машине** — дефолт команды подключения

непроверенный эксперимент

project — **в репозиторий, для всех**

проверенный сервер — конфигурация становится кодом, изменение видно в ревью

user — **глобально во всех моих проектах**

личные инструменты, не специфичные для проекта

Дефолт команды подключения кладёт сервер в local, не в project. Если про это не знать — получится разговор «я подключил, всё работает» — «а у меня нет», и оба правы.

Почему галочки прощёлкивают — и что бывает дальше

Права у ключа спрашивают единственный раз — в момент выпуска: сервис перечисляет действия и спрашивает, какие из них вы этому ключу доверяете.

ХОД, КОТОРЫЙ ДЕЛАЮТ ПОЧТИ ВСЕ	ЧТО В НЁМ СТОИТ ДОРОГО
проставить все галочки подряд: «вдруг потом понадобится, не перевыпускать же ключ»	список операций агента на три строки, а ключ выдан на все: вычеркнутое на разборе вернулось целиком
положить ключ под руку — в файл рядом с конфигурацией	файл уезжает в репозиторий, в историю коммитов и в обзор правок. Дом ключа между запусками — хранилище секретов: видно имя, не значение
понадеяться, что ваш ключ никто не ищет	ключи в публичных репозиториях ищут и находят работающими; один из поставщиков моделей сторожит свои сам и блокирует засвеченный за секунды — по рассказу студентов прошлого потока
вспомнить про права, когда ключ уже где-то всплыл	права меняют на стороне сервиса, и задним числом они не отменяют ничего: всё, что ключ успел, он успел

Сторож поставщика закрывает один канал из многих — приватную утечку, журнал сборки и снимок экрана он не видит. Узкий ключ, который утёк, остаётся единственным, что в вашей власти после выпуска.

Собранный файл — и порядок при конфликте имён

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_PAT}" }
    }
  }
}
```

- `mcpServers` — корень: все серверы этой области. `"github"` — имя сервера: под ним агент зовёт инструменты, `mcp__github__…`.
- `"command"` и `"args"` — чем запускается и что именно: пакет самого сервера.
- `"env"` — окружение процесса; `${GITHUB_PAT}` — подстановка, не сам секрет.



Подключено — не значит, что агент туда ходит

«Подключено» отвечает за процесс: поднялся, ответил на рукопожатие, отдал список своих инструментов. Про то, обращался ли к нему агент в вашей задаче, в этой отметке нет ничего.

- **Доказательный вызов.** Назвать сервер в запросе по имени: «используя сервер трекера, выведи список открытых задач, затем подготовь черновик по каждой».
- **Признак в выводе.** Вызов инструмента подписан именем сервера.

Отсутствия ошибок мало. Ошибки не будет и тогда, когда сигналов не приходит вовсе, — следить надо не только за тем, есть ли сбой, но и за тем, есть ли связь с тем, за кем следите.

Постоянная плата одинакова: своё, чужое, локально, удалённо



Постоянную плату задаёт число инструментов. Имена инструментов каждого сервера и инструкция о нём висят в контексте всегда. Свой сервер, чужой локально, чужой удалённо платят её одинаково: транспорт и авторство значения не имеют. Свободны от неё прямой вызов API и команда вроде `gh` — у них нет списка возможностей, висящего в контексте между вызовами.

К себе. Сколько серверов подключено у вас прямо сейчас и сколько инструментов они описывают? Эту плату вы вносите в каждой сессии — даже в ту, где агент за день не сходил ни к одному.

Проверить у себя: не выставлен ли `ENABLE_TOOL_SEARCH=false`, и что реально показывает карточка конкретного сервера в сессии — команда не устаревает, в отличие от самой цифры.

Чужая инструкция в issue заставила агента опубликовать приватный код

Вернёмся к экрану со-сборки. Мы вычеркнули строку «все репозитории, включая приватные» — вот ровно тот случай, где это решило дело.

Смертельное трио (Simon Willison, 16.06.2025): доступ к приватным данным + обработка недоверенного внешнего контента + канал наружу. При всех трёх атакующему достаточно написать текст, который агент прочитает в рамках обычной работы.

СЛОЙ	ЧТО ПРОИЗОШЛО	БАЗА СРАВНЕНИЯ
1 · эксфильтрация в связке (Invariant Labs, май 2025)	Агент с токеном на все репозитории пользователя, включая приватные. Атакующий создаёт в публичном репозитории issue с инструкцией собрать данные о других репозиториях и опубликовать здесь. Разработчик просит агента разобраться с открытыми issue — агент честно читает их, инструкция попадает в контекст как задача, читает приватные репозитории, публикует их содержимое в автоматически созданном публичном PR	уязвимости не присвоен номер — архитектурный класс связки прав, не брешь кода. Убрать строку «все репозитории, включая приватные» — читать нечего, сценарий не проходит вообще
2 · без модели вообще (CVE-2026-61742, DBHub, CVSS 9.3 из 10)	HTTP-транспорт по умолчанию слушает без аутентификации, защита от межсайтовых запросов обходится DNS rebinding	по вторичному техническому разбору (Strix.ai) — два HTTP-запроса и ноль учётных данных до захвата; то же трио здесь не единственный путь к утечке — сервер сам может быть незащищённым сетевым сервисом

Когда MCP нужен, а когда не нужен

Есть доступ наружу, и есть чем доказать, что агент им пользуется. Следующий вопрос — во что обходится сама подключённость, пока к серверу никто не обратился.

MCP НУЖЕН, КОГДА	MCP НЕ НУЖЕН, КОГДА
доступ нужен регулярно, из нескольких сессий — разового снимка данных недостаточно	доступ разовый или редкий, и снимка данных на сегодняшний день достаточно
вы готовы поддерживать ещё один постоянно работающий процесс с доступом к секретам	такой готовности нет — тогда разовый вызов надёжнее процесса, который никто не обслуживает
операций больше одной, список операций растёт, и агент сам решает, к какой обратиться — за этот список и вносится постоянная плата	задача сводится к одному известному вызову CLI: список возможностей здесь лишний, и тридцатидвухкратная разница по токенам никуда не денется
подключение не достраивает третий элемент смертельного трио	подключение достраивает третий элемент смертельного трио — доступ к приватным данным и обработка чужого текста уже есть, остаётся канал наружу; это стоп-критерий

2

Он открыл сессию в своём репозитории и ещё ничего не спросил

Сессия открыта, ни одного вопроса агенту он ещё не задал. Часть контекста уже занята: четыре сервера из проектного файла описали себя при старте, и за всю задачу понадобится один.

MCP

Субагент

Один сервер его. Ещё три дописал коллега

Проектный файл подключения: трекер задач — его собственный; браузер, база данных, сервер документации — коллеги, под свою работу. Область проектная, её видит каждый, кто работает с репозиторием.

- **Что он заметил.** На длинной правке агент сжал разговор заметно раньше обычного и переспросил то, о чём они договорились в начале.
- **Из чего набрано.** Он посмотрел, что лежит в контексте до его первого вопроса: системная часть, файлы инструкций проекта, описания инструментов четырёх серверов.
- **Что нашлось в чужих строках.** У двух серверов стоит `alwaysLoad: true` — отложенная загрузка описаний для них отменена, все их инструменты описываются при старте. Коллега поставил флаг, чтобы «инструменты не терялись».
- **Цена.** Час на то, чтобы заново проговорить решения, потерянные при сжатии. Одна правка успела уехать в ветку по решению, которое он к тому моменту уже отменил.

За всю задачу он обратился к одному серверу из четырёх. Плата внесена за четыре.

Четыре сервера, понадобился один. Что меняем?

У кого в рабочем проекте подключено больше трёх серверов? А к скольким вы обращались за последнюю рабочую неделю?

Назовите оба числа — и скажите, что вы ставите первым шагом и что добавляете к этому следующим.

- ничего — контекст большой, хватит
- отключить серверы, которыми не пользуетесь
- оставить подключёнными, но получать описания по требованию
- держать набор серверов под задачу, не общий проектный
- чаще сжимать разговор

Сервер, которым не воспользовались, стоит столько же

ВАРИАНТ	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
ничего — контекст большой	оставляет всё как есть: на короткой задаче этого и не заметно	плату вы уже внесли, и она держится весь сеанс: на длинной задаче сжатие приходит раньше, и часть принятых решений уходит в пересказ
отключить серверы, которыми не пользуетесь	снимает плату целиком: у неподключённого сервера в контексте нет ни имён, ни инструкции о себе	требует знать заранее, что понадобится; отключённый нужный возвращает ту работу руками, с которой начинался первый случай блока
получать описания по требованию	оставляет при старте имена инструментов и инструкцию сервера; полную схему модель подтягивает, когда инструмент ей понадобился	это поведение по умолчанию — значит ваше действие здесь проверочное: не отменил ли его флаг у сервера или переменная окружения
держать набор серверов под задачу	личная область подключения даёт набор под вашу работу, общий проектный файл при этом остаётся командным	заводит вторую запись о том, что подключено: расхождение личного и проектного набора придётся держать в голове
чаще сжимать разговор	освобождает место, занятое историей разговора	описания инструментов сжатием не уходят: они приходят заново в каждый контекст, отдельно от истории

Из сочетания колонок: плата идёт за подключённость — сервер, к которому за сессию не обратились ни разу, стоит столько же, сколько рабочий. Поэтому работают две меры, и в таком порядке: сначала проверить, что описания приходят по требованию, — это ничего не стоит и часто уже включено; потом разобрать набор серверов под задачу — это работа, и её делают один раз.

Что лежит в контексте до вашего первого вопроса

ОТКУДА	ЧТО УЖЕ ЗАНЯТО — ПО УМОЛЧАНИЮ
клиент	системная часть: инструкции клиента и описания его собственных инструментов
репозиторий	файлы инструкций проекта — те, что вы собрали за три занятия
каждый сервер	имена всех его инструментов
каждый сервер	инструкция сервера о себе, обрезанная по 2 048 символов

по требованию:

полную схему инструмента модель подтягивает, когда решила, что он понадобится

отсрочку отменяют:

`alwaysLoad: true` — у отдельного сервера: все его схемы при старте, независимо от общего режима

`ENABLE_TOOL_SEARCH=false` — в окружении: отсрочки нет ни у кого

тогда счёт идёт по полным схемам: 93 определения — 42 000–55 000 токенов,
около 21% контекста в 200 000 (замер, 2026 — та же цифра, что у цены подключения)

Описания — по требованию. Набор — под задачу

- `.mcp.json`: у сервера снят `alwaysLoad: true` — описания его инструментов приходят по требованию. Флаг оставляют одному инструменту, который нужен на каждом шаге, и записывают рядом, почему.
- Окружение проекта: `ENABLE_TOOL_SEARCH` не выставлен в `false`. Пороговое значение вида `auto:10` — компромисс: пока определения занимают меньше названной доли контекста, они грузятся сразу.
- Команда запуска сервера: только нужные наборы инструментов — --toolsets `repos,issues` вместо полного набора. Проверяйте флаги по текущей версии сервера: у официального сервера GitHub режим выдачи наборов по требованию был и снят в мае 2026.
- Личная область подключения: серверы под свою работу — там. Общий проектный файл остаётся командным, и чужие три сервера из него никто не выбрасывает.

Проектный файл говорит, что подключено у всех. Личная область — что подключено у вас. Режим загрузки описаний решает, во что это обходится до первого вопроса.

Описания убрали со старта. Серверы оставили

1. **Автор протокола назвал это проблемой.** «Определения инструментов перегружают контекст» — так открывается его собственный разбор. Ход взамен: представлять серверы кодовым интерфейсом, который агент читает по требованию; в разобранном в статье примере — 150 000 токенов против 2 000. Серверы остались на месте (Anthropic, инженерный блог, 04.11.2025).

2. **Счёт на пяти серверах.** Набор GitHub, Slack, Sentry, Grafana, Splunk занимает около 55 000 токенов определений до начала работы; загрузка по требованию снимает больше 85%, подтягивая 3–5 нужных инструментов. Там же: точность выбора падает, когда доступных инструментов больше 30–50 (документация того же производителя, обращение 07.10.2026).

3. **Сократили собственный набор.** Встроенных инструментов было 40, оставили 13 основных — и замерили: успешность на двух наборах задач выросла на 2–5 процентных пунктов. Предел площадки — 128 инструментов на запрос (GitHub, инженерный блог, 19.11.2025).

4. **Попробовали и отказались.** Подгружать инструменты по требованию пробовали и отказались: смена набора посреди работы обнуляет кэш и путает модель на прошлых шагах. Ход взамен — маскировать лишние, оставляя определения на месте (Manus, 18.07.2025).

Посмотрите, в каком режиме описания грузятся у вас, и держите набор под задачу. Одинакового ответа у отрасли нет: последняя строка прямо обратна трём предыдущим.

Сколько серверов держать подключёнными

В конфигурации теперь пять файлов: к четырём, с которых мы начали занятие, добавился `.mcp.json`. Он дал агенту руки — под правами, которые мы вычеркнули сами, с проверкой, которую умеем делать, и по цене, которую теперь видим. Контекст у агента при этом один и конечный, и описания инструментов в нём — одна из статей. Следующий вопрос — что в этот контекст всё равно не помещается, когда работы становится больше.

ДЕРЖАТЬ ПОДКЛЮЧЁННЫМ В ПРОЕКТНОМ ФАЙЛЕ

- сервер нужен в большинстве сессий этого проекта: им пользуется команда
- описания его инструментов приходят по требованию, и это проверено
- наборов инструментов у него столько, сколько нужно задаче; остальное сервер держит выключенным

НЕ НУЖНО НИКОГДА

- держать в общем проектном файле то, что нужно одному человеку: для этого есть личная область
- ставить флаг принудительной загрузки описаний, не записав рядом причину: через месяц это чужая строка без объяснения
- считать, что большой контекст снимает вопрос: точность выбора инструмента падает после 30–50 доступных, и видно это по результату: счёт молчит
- полагаться на флаг, найденный поиском: у сервера его могли снять — проверяется по описанию текущей версии

3

Сорок файлов ради четырёх нужных мест

Что править — известно. Где править — лежит в сорока файлах, и прочитать их надо туда же, где эта правка уже объяснена.

MCP

Субагент

Агент, субагент и контекст каждого

Контекст один на сессию — те же источники, что считал прошлый блок занятия

харнесс	**системная часть** какие инструменты есть, как отвечать, что делать запрещено
репозиторий	**файлы инструкций проекта** CLAUDE.md и AGENTS.md, всеми уровнями — от корня до папки
подключённые серверы	**имена инструментов и инструкция сервера** по всем серверам, до первого вашего вопроса
вы и агент	**разговор → растёт** задача, ответы агента и содержимое каждого прочитанного файла

Три источника из четырёх приходят ДО вашей первой строки — вы их не писали и не выбирали. Четвёртый растёт с каждым файлом, который агент открыл: поэтому контекст кончается раньше, чем кажется по длине разговора.

Кто этот контекст держит — и что из чужого контекста возвращается в ваш

СЕССИЯ, В КОТОРОЙ ЛЕЖИТ РАБОТА

агент — с ним вы разговариваете

его контекст — все четыре полосы сверху:
системная часть · инструкции проекта · описания серверов · разговор, где объяснена задача

Здесь лежит правка.
Сюда же нужен результат чтения —
иначе правку нечем продолжить.

Прочитаете сорок файлов здесь — четвёртая полоса вытеснит объяснённую задачу.

задача → текстом

итог ← **сам**

СУБАГЕНТ, ПОЗВАННЫЙ АГЕНТОМ

отдельный исполнитель под одну задачу

свой контекст:
те же три верхние полосы, а вместо разговора — одна задача, присланная текстом.
Вашей переписки тут нет.

Сорок файлов ложатся сюда и закрываются вместе с субагентом.

Обратно уходит только итог — и уходит без вашего участия.

ВТОРАЯ СЕССИЯ, КОТОРУЮ ОТКРЫЛИ ВЫ

свой агент, свой контекст

сорок файлов ложатся сюда — рабочий контекст чист.
Но и ответ остаётся здесь: канала в рабочую сессию нет.

Переносите вы — руками и своим пересказом.

Агент, который правит, видит только ваш пересказ.

через вас: прочитать, понять, пересказать заново

Субагент возвращает итог сам — в тот контекст, где лежит работа. Из второй сессии его переносите вы. Поэтому вторая сессия освобождает контекст, но не продолжает работу.

Приём переносим: отдельный субагент со своим контекстом есть у нескольких продуктов для работы с кодом. Запись у каждого своя — общей спецификации нет ни у одного.

Субагент целиком, одним экраном

1 ГДЕ ОБЪЯВЛЯЕТСЯ

```
# .claude/agents/<имя>.md
name: diff-reviewer
tools: Read, Grep, Glob
```

`description:` — когда звать, когда нет — решает автовыбор

`тело файла` — системный промпт субагента целиком

2 КАК ПОЗВАТЬ — и кого позвали на самом деле

`@agent-<имя>`
вызов явно — гарантирует именно этого субагента

`ничего не писать`
модель решает сама, по description

`<имя>(задача)`
строка в чате; general-purpose вместо имени — не долетело

3 ЧТО НА ВХОД

системный промпт — тело файла субагента — не промпт харнесса целиком
задача — сформулирована вызывающей сессией, не дословно
файлы инструкций — иерархия CLAUDE.md/AGENTS.md, все уровни
снимок репозитория — состояние на момент старта

✗ **переписки сессии, которая позвала субагента — нет**

4 ЧТО НА ВЫХОД

итог — вызвавшей сессии уходит только финальный текст

✗ **вызовы инструментов**
ни один не виден вызвавшей сессии

канал в одну сторону: задача → субагент → только итог

5 ПРАВА — что значит каждая запись tools:

tools: не указано вовсе
субагент получает ВСЕ инструменты, доступные субагентам — противоположность «без инструментов»

tools: [] — явно пуст
субагент без единого инструмента — но живой

запись не резолвится
харнесс не запускает субагента вовсе

6 ЧЕМ ОПЛАЧИВАЕТСЯ:

мультиагентная работа ≈ в 15× дороже обычного чата (оценка поставщика харнесса; измерений на выборке за ней нет)

Нашёл все четыре места. Правку начал заново

1. Тот же разработчик, что заводил здесь хук и скилл. Сцена другая: продукт, где он работает каждый день, — тарифы, участники, два года миграций. Задача дня — завести тариф «Команда»: двадцать участников вместо пяти.

2. Сессия открыта с утра. Минут пятнадцать он объясняет агенту задачу: как тариф называется, кому доступен, что делать со старыми аккаунтами, какие случаи проверить. Правку они делают здесь же.

3. Прежде чем править, нужен список мест, где лимит участников вообще проверяется. Пропустишь одно — новый тариф пройдёт конфиг и упрётся в проверку, про которую никто уже не помнит.

4. Агент ищет и находит четыре: middleware при добавлении участника, конфиг тарифов, отдельная миграция для аккаунтов до смены тарифной сетки, клиентский компонент с той же проверкой на случай недоступности сервера. Миграций за два года пятнадцать, лимит менялся в трёх — листать приходится все. Открытых файлов больше сорока, чтения — минут двадцать.

5. «Теперь правь по этому списку» — и агент переспрашивает то, что ему объясняли утром: как называется тариф и сколько в нём участников. Объяснять приходится второй раз.

Полчаса ушло на то, чтобы сказать уже сказанное и вернуться туда, где он остановился. Список при этом верный, и чтение шло быстро: нужное влезло в контекст целиком и вытеснило работу, ради которой сессию открывали. Это второй промах из начала занятия, предъявленный целиком.

Что делать с этим чтением

У кого из вас была правка, в которой найти места стоило дороже, чем их поправить? Сколько файлов пришлось открыть?

> *Здесь мест четыре, файлов сорок.*

Выберите карточку — и назовите вслух, за счёт чего она выигрывает.

- прочитать все сорок файлов самому, в этой же сессии
- открыть вторую сессию, спросить там и перенести список руками
- позвать коллегу, который давно работает с тарифами
- поручить субагенту: прочитает в своём контексте и вернёт список мест
- править по памяти, без поиска

Быстрее не стало — те же сорок файлов

ВАРИАНТ	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
прочитать самому, в этой же сессии	полный контроль над тем, что и в каком порядке читается	контекст сессии занимает содержимое сорока файлов, из которых для правки нужны четыре
вторая сессия	рабочий контекст остаётся чистым: сорок файлов ложатся в чужой	список придётся переносить руками, и править агент будет по вашему пересказу — условие про старые аккаунты выпадает из пересказа первым
позвать коллегу	единственный вариант, который может ответить за минуту — если коллега свеж в теме	единственный вариант, который зависит от чужого расписания: решает, свободен ли человек прямо сейчас
поручить субагенту	сорок файлов открываются и закрываются в отдельном, одноразовом контексте; список приходит обратно сам, в ту же сессию	быстрее не становится: субагент открывает и читает все сорок файлов, время чтения то же
править по памяти	мгновенно	четвёртое место — клиентскую проверку — помнит не всякий из тех, кто её писал

Вывод. Двадцать минут чтения из сцены остаются двадцатью минутами: сорок файлов — это сорок файлов, где бы их ни читали. Меняются двое. Первое — чей контекст их держит: контекст субагента закрывается вместе с ним. Второе — куда возвращается список: от субагента он приходит сам, в ту сессию, где лежит правка, из второй сессии его переносите вы.

Пример есть, измерения на выборке — нет

> ***«The subagent read 6,100 tokens of files. You got a 420-token result. That's the context savings.»*** *— документация харнесса об устройстве контекста, один показательный сценарий.*

Разница на порядок — но это один вымышленный пример автора документации, не статистика по множеству реальных прогонов.

Честный пробел. *Сколько контекста реально экономит делегирование по сравнению с тем же объёмом работы в основной сессии, по измерению на выборке сессий, — не нашлось ни у одного источника. Это число восполняет пробел дидактическим примером, не статистикой.*

Чистый контекст — не от всего подряд

> ***«Subagents inherit every MCP tool definition from the parent session, which can fill their context window before the first turn. Disable MCP servers you are not using before spawning subagents.»*** *— документация харнесса об устранении ошибок.*

Это третья полоса со схемы базы — «описания подключённых серверов». В контекст субагента она переезжает целиком. Для нашего поиска по лимиту участников: если в сессии подключены трекер задач и сервер документации из прошлого раздела занятия, субагент, которому нужно прочитать сорок файлов кода, получает их описания бесплатно.

Фокус из прошлых двух слайдов — про содержимое репозитория. За тем, что подключено серверами, следят отдельно: тем же списком инструментов субагента, который стоит пятым блоком на одностраничнике ступени.

Что остаётся в контексте после ответа

Что мы заводим ради контекста — решаем по тому, что останется в контексте после ответа, и не по тому, сколько файлов пришлось открыть. Интуиция говорила: отдельный исполнитель прочитает быстрее. Проверка показала: сорок файлов остаются сорока файлами и тем же временем, а меняются двое — чей контекст их держит и куда сам возвращается итог.

- Делегировать чтение имеет смысл, когда из десятков файлов нужно вынести короткий вывод: факт, да/нет, список мест в несколько строк, по которому дальше и работают. Сорок файлов поиска, четыре файла правки — ровно этот случай.
- Если дальше работать придётся со всем прочитанным объёмом — рефакторинг тех же сорока файлов, — читать их придётся в любом случае; делать это в своём контексте тогда входит в саму работу, потому что с этим содержимым дальше работать вам.

К себе. Назовите про себя место в своём проекте, где перед правкой приходится искать по десяткам файлов, а находятся три-четыре строки. Назвали — вот первый кандидат на делегирование. Если править вы будете все эти же десятки файлов, кандидат не он.

4

Задача пришла одной фразой. Чем ускоряем?

Одна фраза, один срок. Средств ускорить больше одного, и выбирает между ними сама задача.

MCP

Субагент

Задача к понедельнику. Раскладываем вместе

«К понедельнику форма должна быть готова к рекламе.»

Пятница, вечер, это вся постановка. Форма заявки на лендинге прошлых занятий: разметка — `index.html`, проверка — `src/validate.js`. Спросить не у кого.

- **Ход 1.** Выпишите, из каких кусков состоит эта задача.
- **Ход 2.** По каждому ответьте: нужен ли ему итог другого, лезет ли он в тот же файл, повторяется ли после остальных.
- **И одной строкой** — сколько исполнителей вы завели бы на эти куски.

ХОД 1 · ЧТО В ЗАДАЧЕ ЛЕЖИТ	ХОД 2 · ОТ ЧЕГО ЭТО ЗАВИСИТ
поле «телефон» и проверка формата	ни от чего
сообщения об ошибках — к одному виду	тот же файл, что у поля: `src/validate.js`
текст согласия на обработку данных	ни от чего
проверка формы на узком экране	после поля: пока поля нет, проверять нечего
прогон перед выкладкой: ветка, тесты, сборка	ни от чего, и повторяется после каждого
письмо-подтверждение заявителю	ни от чего, кода не касается

Независимых кусков — три из шести. Один ждёт поля. Один лезет в тот же файл. Один повторяется после каждого из остальных.

Чем ускоряем?

Кто из вас уже разгонял задачу разом, а не по очереди? Чем именно — несколькими сессиями, субагентами, проверкой, которая гоняется сама?

Теперь то же на этой задаче: выберите карточку — и назовите кусок, на котором ваше средство работает.

- разложить и делать куски по очереди
- вести куски в нескольких сессиях
- раздать куски субагентам и запустить разом
- вынести повторяемое в проверку, которая гоняется сама
- сузить задачу — к понедельнику нужно не всё

Параллельность лежит в задаче

СРЕДСТВО	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
разложить и делать куски по очереди	один предсказуемый порядок, ничего ни с чем не сталкивается	три независимых куска ждут друг друга без причины: ожидания складываются
вести куски в нескольких сессиях	куски идут одновременно, у каждой сессии свой контекст	две сессии в одной папке правят одни и те же файлы — у нас это поле и сообщения об ошибках, один `src/validate.js`
раздать куски субагентам и запустить разом	одновременность даёт харнесс, настраивать её не нужно; у каждого субагента свой контекст и короткий итог	берёт параллельность там, где она есть: на поле и сообщениях два субагента увидят разные версии одного файла
вынести повторяемое в проверку, которая гоняется сама	прогон перед выкладкой уходит из очереди совсем — его делает не человек	писать проверку надо заранее и один раз; в пятницу, когда задача уже пришла, она ускорит только следующую такую задачу
сузить задачу	для рекламы нужны поле и согласие — два куска из шести	решает это не исполнитель: сузить может тот, кто отвечает за срок

Поворот. Ни одно из пяти средств не ускоряет все шесть кусков. Параллельность лежит в задаче, и в этой её ровно на три куска: субагенты берут её там, где она есть, и умножают ошибку там, где её нет — на куске, который лезет в чужой файл, и на куске, который стоит в очереди. Поставьте рядом два числа: исполнителей, которое вы назвали вслух, и независимых кусков, которое вышло из раскладки.

Фон, не очередь — и предел на двадцать разом

- **Фон, не очередь.** С определённой версии харнесса параллельный режим включён по умолчанию в интерактивной сессии: субагент уходит в фон, основная сессия не ждёт его, прежде чем продолжить.
- **Предел — на одновременность.** По умолчанию одновременно может работать не больше двадцати субагентов; попытка завести двадцать первого падает отдельной, явной ошибкой. Число меняется переменной окружения.

Общего предела на то, сколько субагентов можно вызвать за всю сессию, нет.

Оркестратор и дочерние сессии



Так работает автор курса. Дочерние сессии — полноценные, со своим контекстом; каждая дробит свою часть дальше сама.

ЧТО ДАЁТ ДОЧЕРНЯЯ СЕССИЯ	ЧЕГО ОНА СТОИТ
в неё можно прийти руками: посмотреть, что она успела, дописать задачу, поправить направление	оркестратор не узнаёт об итоге сам — он узнаёт тогда, когда сходит и проверит
она сама запускает субагентов: ярусов становится три, и работа берётся крупнее	нужна обвязка: сессии рядом не поднимаются и не опрашиваются сами по себе
контекст у неё свой целиком, как у любого обычного запуска	и стоит она как обычный запуск — по счёту за токены и по памяти машины

Маленький кусок — субагенту. Крупный кусок, за которым придётся прийти руками, — дочерней сессии.

Сессии стоят памяти машины

Замер автора курса на своей машине: облачная виртуалка, 16 ГБ памяти, обычные сессии без тяжёлых прогонов внутри.

ЧТО ПОМЕРЕНО	ЧТО ИЗ ЭТОГО СЛЕДУЕТ
с двадцати одновременных сессий машина начинает спотыкаться	порядок величины — 1–2 ГБ на одну сессию, и это в покое, без того, что сессия запустит сама
на тридцати-сорока, смотря по объёму, она перестаёт отвечать	десяток сессий на такой машине — рабочий предел, который считают до запуска
подъём ещё одной сессии стоит столько же, сколько подъём первой	предел инструмента — двадцать субагентов разом внутри одной сессии; память машины считается по числу самих сессий и может кончиться раньше

Язык, на котором написан сам агент, здесь виден: клиент, собранный на Rust, заметно легче собрата на JavaScript. Кратность автор оценил на глаз.

Это замер одного человека на одной конфигурации. Порядок величины стоит перемерить у себя — на своей памяти, своём числе сессий и своём агенте.

Через неделю: справочник и форма, настроенные порознь

- Через неделю — другая работа: настраиваемая платформа, та, где справочники, формы, права доступа и скрипты заводят настройкой. Параллелить там особенно соблазнительно, потому что каждый кусок выглядит самостоятельным экраном.
- Два куска уходят двум субагентам одновременно: один заводит справочник, второй настраивает форму, которая этот справочник показывает и проверяет. Общего описания задачи никто не пишет — оно кажется лишним, раз куски видны на экране.

Оба возвращают сделанную работу, и оба правы: справочник заведён, форма настроена. Вместе они не работают — поля в справочнике названы и составлены одним, форма ждёт другого. Сводить это приходится руками, и сведение занимает больше, чем заняла сама настройка.

Условие, которое никто не написал

ХОД	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
отдать оба куска одному субагенту	один исполнитель видит оба конца и держит состав полей согласованным сам	отдаёт параллельность там, где она была настоящей и бесплатной; на большой задаче один исполнитель становится узким местом
дать каждому широкий доступ — пусть сам разберётся в соседнем куске	иногда срабатывает: исполнитель находит соседнюю настройку и подстраивается	широкий доступ означает широкий контекст и широкие права; прочитать соседний кусок ещё не значит понять его так же
общее описание задачи, которое оба читают до начала	состав общего назван один раз для обоих, и расхождение видно до работы	описание надо написать и поддерживать: появляются циклы чтения, правки и согласования, и это настоящая плата
завести третьего субагента-координатора	сверяет две готовые работы между собой	добавляет слой координации к двум уже разошедшимся; сводить всё равно придётся, просто чужими руками

Вывод. В раскладке условие назвали вслух: сказали, какие куски независимы и почему. Здесь общего описания не написал никто, и додумать его не мог ни один из двоих — целиком задачу не знал никто. Работает третий ход, и работает он тем же, чем работала раскладка.

Масштаб, не единичный случай

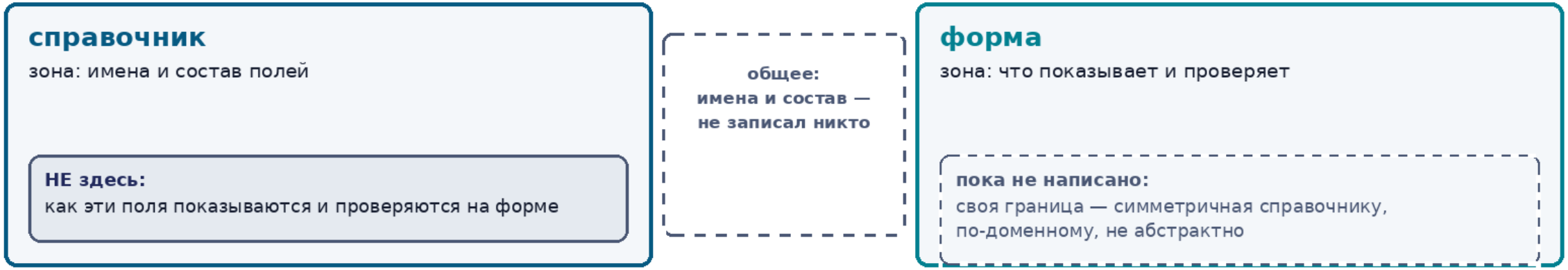
Для наших двух субагентов. Тот, кто заводил справочник, и тот, кто настраивал форму, работают без третьего субагента, сверяющего их согласованность. Вот что об этом говорит измерение на масштабе, не на одном случае.

ИСТОЧНИК	ТЕЗИС	СИЛА ДОКАЗАТЕЛЬСТВА
Cemri и др., «MAST», arXiv:2503.13657 (NeurIPS 2025 D&B)	От 41 до 87% размеченных трасс (в зависимости от фреймворка) — провальные; три категории причин: спецификация задачи и роли ≈42%, рассогласование между агентами ≈37%, верификация результата ≈21%	сильное: ручная разметка 1600+ трасс по 7 фреймворкам, межэкспертное согласие к=0,88
Kim, Gu, Park и др. (Google Research), arXiv:2512.08296	На последовательной задаче схема проигрывает одноагентной той же задачи на 39–70%; без координатора число ошибок усиливается в 17,2 раза относительно одноагентной системы на входе, с централизованной проверкой — в 4,4 раза	сильное: контролируемый эксперимент, 260 конфигураций, 6 бенчмарков, 5 архитектур

Честный пробел. *Проверить рассогласование двух параллельных субагентов на репозитории курса или на машине студента нечем — ни один инструмент проверяемости занятия это не демонстрирует. Утверждение подтверждено исследованием, не командой на занятии.*

Что пишут в общем описании: границу в обе стороны

Общее описание держит куски вместе: граница каждого — в обе стороны



Граница делает расхождение видимым тому, кто читает описание до работы — сам стык между кусками она не убирает.

Справочнику нужна строка про то, где он кончается: «имена и состав полей — здесь; как они показываются и проверяются на форме — не здесь». Форме — симметричная: «показываю поля по именам из описания; состав полей не завожу».

Пока расхождение видно до работы, оно стоит двух строк. Прочитать описание перед началом остаётся решением человека.

Субагент — одно средство из пяти. Где он оправдан

Отраслевой разворот, как метка границы. Одна команда год назад полностью отвергала мультиагентность, называя неструктурированный «рой» отвлекающим фактором. Десять месяцев спустя она же вернулась к двум узким паттернам — ревью-цепочка (отдельный субагент находит в среднем два бага на правку, из них около 58% серьёзных) и связка из двух разных топ-моделей на сложных сценариях. «Рой» без границ она по-прежнему называет отвлекающим фактором.

Параллельность лежит в задаче. Догадка считала исполнителей: шесть кусков — шесть исполнителей. Раскладка назвала условие: независимых кусков три, и каждому из остальных нужно своё средство. Нашему разработчику неверный счёт стоил недели работы по очереди — включая те куски, которые спокойно делились. Сведение разошедшихся настроек стоило дольше самой настройки; доверие к приёму стоило недели.

СУБАГЕНТ ОПРАВДАН, КОГДА

кусков несколько и они независимы — шаг одного не нуждается в выводе другого

у каждого куска своя узкая зона и короткий итог — справочник и форма ровно этот случай, когда общее между ними записано до начала

ДЕШЕВЛЕ ДРУГОЕ СРЕДСТВО, КОГДА

шаги зависимы: разбиение умножает ошибку — в 17,2 раза без координатора, в 4,4 даже при централизованной проверке

общего между кусками не записал никто: две честные работы складываются в готовность, которой нет

кусок повторяется после каждого из остальных: его место в проверке, которая гоняется сама

кусок к сроку не нужен вовсе: такой сужают, а ускорять в нём нечего

5

Две сессии в одной папке. Обе отчитались успехом

Сессий у вас уже несколько, а исполнителей в одной папке почти всегда больше одного — и это нормально. Отдельную копию репозитория вы дадите сессии сразу и захотите дать субагенту. Поле изоляции у субагента есть, включают его редко, и это решение. Решаем, где разводить копиями и где только зонами.

MCP

Субагент

Вторая сессия завела ветку. Первая этого не заметила

На ветке `fix/email-pattern` лежит начатая правка: регулярное выражение адреса в `index.html` поправлено, сообщение в `src/validate.js` — ещё нет. Коммита пока нет.

1. Приходит вторая задача: поднять зависимость, из-за которой ругается сборка. С проверкой адреса она не пересекается ни одним файлом. Ждать первую сессию незачем, и разработчик открывает вторую — в том же каталоге, на том же репозитории.
2. Вторая сессия начинает с собственной ветки: `git checkout -b chore/bump-deps`. Каталог один, ветка в нём одна на всех — и незакоммиченная правка адреса переезжает на новую ветку вместе с каталогом.
3. Вторая сессия заканчивает работу и забирает в коммит всё, что видит в каталоге. В коммит с зависимостью попадает недоделанная проверка адреса — для этой сессии она просто часть состояния файлов.
4. Первая сессия продолжает по тому, что прочитала раньше: дописывает сообщение, прогоняет тесты, докладывает, что правка готова. Её ветки под ней больше нет — каталог стоит на ветке зависимостей.

Обе сессии доложили об успехе, и обе были правы по отдельности. Утром выяснилось: в ветке зависимостей лежит чужая незаконченная правка, в ветке адреса — ничего, и полчаса ушло на вопрос, почему тесты формы падают там, где формы никто не касался.

Обе задачи нужны. Что здесь меняют?

Было ли у кого, что два запуска дерутся за одну папку — ветка не та, правка перекрыта, а странного никто не делал?

> *Задачи независимы, обе сессии нужны разом, каталог один.*

Выберите карточку — и обоснуйте её рядом с остальными четырьмя.

- коммитить перед каждым переключением
- работать по очереди, в одной сессии
- вместо второй сессии — субагенты в первой
- каждой сессии — свою копию репозитория
- развести по файлам и договориться

Копия разводит сессии. Внутри сессии разводят зоны

ХОД	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
коммитить перед каждым переключением	возвращает потерянное: есть к чему вернуться, и чужой коммит виден в истории	ветка в каталоге остаётся одна на всех; о столкновении узнаёте после него
работать по очереди, в одной сессии	снимает столкновение целиком: одна работа в каталоге, одна ветка, один исполнитель	вторая задача стоит, пока идёт первая — та самая очередь, от которой ушёл прошлый кейс
вместо второй сессии — субагенты в первой	снимает спор о ветке: субагенты работают в той же ветке, и задачу не нужно объяснять заново	каталог у них общий по устройству, поэтому спор о файле остаётся на месте
каждой сессии — свою копию репозитория	даёт каждой свои файлы и свою ветку при общей истории: переключение у одной под руками другой ничего не меняет	копию нужно поднять как рабочее место — зависимости, файлы вне версий; сводить работу всё равно веткой
развести по файлам и договориться	работает там, где копий не бывает: внутри одной сессии, между её исполнителями	держится на том, что зоны действительно не пересекаются — один общий файл отменяет договорённость

Вывод. Копия репозитория разводит сессии: своя папка, своя ветка, общая история. Внутри одной сессии разводить нечем — исполнители делят каталог по устройству, и защита там держится на раскладке зон. Одно слово «параллельно», два разных ответа, и дальше в кейсе оба.

Своя копия: что в ней своё и что общее

Отдельная рабочая копия — свой каталог файлов и своя ветка; история и удалённый репозиторий те же, что у основной копии. В git это `worktree`. Запуск с флагом --worktree и именем создаёт её под `.claude/worktrees/имя/` на ветке `worktree-имя`; тот же запуск с другим именем в другом терминале поднимает вторую изолированную сессию. Руками то же делает `git worktree add ../проект-правка -b правка`.

СВОЁ У КАЖДОЙ КОПИИ	ОБЩЕЕ С ОСНОВНОЙ КОПИЕЙ
файлы в каталоге — переключение ветки в одной копии не меняет файлы в другой	служебный каталог репозитория: команды из копии пишут в общую историю, поэтому коммит из копии виден везде и переносить работу никуда не нужно
ветка и незакоммиченное состояние	плагины, установленные на уровне проекта
своё рабочее место: зависимости и всё, чего нет в версиях, в копии поднимают отдельно	разрешение, выданное один раз ответом «больше не спрашивать»: оно сохраняется в основную копию и действует во всех

Свежая копия приходит без того, чего нет в версиях: зависимости в ней не установлены, локальных файлов настроек в ней нет. Какие из них переносить в каждую новую копию, задают отдельным списком.

Порядок работы с копиями — строкой в инструкции проекта

- `CLAUDE.md`, раздел `Repository etiquette` — рядом с уже стоящей там строкой «Прямо в `main` не коммитим» добавляются две.
- «Каждая задача ведётся в своей рабочей копии: --worktree <имя> при запуске либо `git worktree add`.» · «Закончив задачу, предложи слияние ветки и уборку копии; незакрытую копию не оставляй.»
- Статус тот же, что у двух остальных сегодняшних артефактов: по плану, коммита нет и не будет.

Почему в файл. Инструкция проекта читается при старте каждой сессии, и порядок работы с копиями достаётся каждой из них одинаково. Договорённость, которая живёт в памяти, достаётся тем сессиям, про которые вспомнили.

Почему с двух концов. Завести копию — делается охотно: без этого не начать. Слить ветку и убрать копию — делается после того, как задача решена, и поэтому забывается. На рабочей машине автора курса нашлось больше сотни старых копий; обнаружилось это, когда кончилось место на диске.

Субагенты в одной папке: что здесь ломается

Субагент стартует в том каталоге, где стоит вызвавшая его сессия, и уйти из него не может: смена каталога внутри субагента не переживает его собственных вызовов. Общая папка у субагентов — штатный режим, в котором они работают почти всегда.

КАК УСТРОЕНО	ЧТО ИЗ ЭТОГО ЛОМАЕТСЯ
каталог у субагента тот же, что у вызвавшей его сессии; инструмент правки есть у всякого субагента, которому его не отняли строкой	два субагента правят одни файлы; тот, что записал позже, перекрывает работу первого — и перекрывает целиком, когда переписывает файл вместо правки участка
состояние репозитория субагент получает снимком на момент своего старта, и снимок по ходу работы не обновляется	субагент пишет по состоянию, которого в каталоге уже нет: создаёт заново файл, которого «не было», или правит строку, которую рядом уже переписали
вызвавшая сессия получает итог субагента; шагов, которыми он получен, она не видит	оба итога приходят успешными, и по ним столкновение не видно — видно его в самих файлах

Так и устроено, и это нормально: субагент работает там, где лежит работа, иначе итог было бы некуда возвращать. Поэтому первая защита здесь — раскладка зон. Изоляция у субагента тоже бывает, и её цена разбирается дальше в кейсе.

Чем это нивелируют

Четыре приёма из пяти — про раскладку и порядок записи, пятый ловит то, что они пропустили. Ни один не требует отдельной копии, и все пять работают там, где её не бывает.

ПРИЁМ
развести зоны до запуска: у каждого исполнителя свой набор файлов
читать файл прямо перед записью, не опираться на прочитанное в начале работы
править участок, не переписывать файл целиком
давать непересекающиеся задачи, а сведение результатов вести отдельным проходом
признак успеха брать из самого файла: перечитать результат, сверить время правки

ЧЕГО ОН НЕ ЗАКРЫВАЕТ
общий файл, нужный двоим; он остаётся отдельной работой, и чьей именно — решают заранее
промежуток между чтением и записью остаётся, и он не нулевой
структурную переделку файла так не сделать — там правка и есть «целиком»
сам этот проход не случается: его кто-то делает, и это видимая работа
ловит столкновение, которое уже случилось, и ничего не предотвращает

Все признаки успеха сошлись

Этот курс собирают так же — несколькими сессиями разом, в одной рабочей копии. Три случая из его собственного журнала, все три записаны в репозитории, открытом перед вами.

СЛУЧАЙ	ПРИЗНАКИ УСПЕХА	ЧТО БЫЛО НА САМОМ ДЕЛЕ
шесть сессий правили одну деку в одном каталоге; соседняя сборка переписала входной файл, пока его читал конвертер	код возврата ноль, построчно «отрисован слайд 44 … 50», «PDF скопирован», верное число страниц	все 54 страницы из 54 пустые, файл похудел с 6,67 до 2,69 МБ. Поймано сравнением времён: вход оказался новее выхода
две сессии правили один файл сборки; чтение файла попало в середину чужой записи	сборка упала, и ошибка указывала на имя, которого «нет»	строка объявления в файле была; через полминуты та же сборка прошла без единой правки с чьей-либо стороны
лекцию собирали параллельно с другой в одной рабочей копии, ветку переключали из обеих сессий	каждая сессия видела свою ветку на месте — в тот момент, когда смотрела	больше семи переключений ветки за одну сессию и около двух часов на восстановление. С переходом на отдельные копии — три, затем четыре параллельные сборки без единого столкновения

Код возврата, число страниц, свежесть файлов — все привычные признаки сошлись, и ни одна сессия по отдельности не сделала ничего неверного. Единственное, что показало правду, — сам результат и время правки.

`git worktree list` — у себя: есть ли у вас хоть одна отдельная копия и сколько сессий сейчас стоят на одном каталоге?

Почему копию субагенту дают редко

Копию субагенту дать можно: одна строка в его описании — и субагент работает во временной копии репозитория; она убирается сама, если субагент ничего не изменил. Поле есть, включают его редко.

Изоляция субагента — выбор под работу. Там, где его правку нормально принять веткой, копия окупается; там, где субагент правит начатое вами, копия отрезает субагента от работы. Напарникам в команде агентов копий не дают вовсе, и предписание для них то же: развести работу по файлам.

КОПИЮ СУБАГЕНТУ ДАЮТ

работа субагента — цельное изменение, которое нормально принять веткой: механический перебор по многим файлам

изменений много, и проверять их удобнее отдельно от вашей собственной правки

готовый приём делит одно крупное изменение на десятки изолированных субагентов разом

КОПИЮ СУБАГЕНТУ НЕ ДАЮТ

субагент работает над тем, что вы начали в своём каталоге: отдельная копия отрежет его от этой работы

правка короткая и нужна здесь же — поднять рабочее место копии дороже самой правки

субагент читает и возвращает итог: столкнуться ему нечем

- **Цена первая — ветка.** Копия субагента отрастает от ветки по умолчанию репозитория, пока отдельную настройку не поменяли. Работы, начатой в вашем каталоге, изолированный субагент просто не видит.
- **Цена вторая — возврат.** Канал назад у субагента один: итоговый текст. Файловые правки остаются в его каталоге, на его ветке, и принести их в вашу работу приходится вам.
- **Цена третья — рабочее место.** В копии оно тоже собственное: зависимости и локальные файлы субагент получает в том виде, в каком их туда положили.

Где у вас проходит граница

Копия разводит сессии, зоны разводят исполнителей внутри сессии. Где кончается одно и начинается другое — у каждой команды по-своему.

Два вопроса, и готового ответа у них нет.

Назовите свой — и назовите условие, при котором он меняется.

1. Что должно быть верно про две работы, чтобы вы согласились вести их в одной папке? И по какому признаку вы заводите отдельную копию вместо работы по очереди?
2. Сколько исполнителей с правом записи вы готовы держать в одной папке разом? Назовите число и признак, по которому остановитесь.

Что встало на место двух промахов

СТУПЕНЬ	ПРЕДОТВРАЩАЕМ / РЕШАЕМ	ЧТО ДЕЛАЕТ	ЧТО НЕ ДЕЛАЕТ / ОГРАНИЧЕНИЕ
Хук	Решаем — правило уже нарушено	Отклоняет один паттерн записи в `main` из шести способов коммита	На остальных пяти молчит; человека с терминалом не видит вовсе
Скилл	Решаем — по сигналу перегрузки файла инструкций	Загружает файл по точному совпадению описания-триггера	Без точного триггера не сработает вообще — содержимого файла до срабатывания в контексте нет
MCP	Предотвращаем на входе, решаем после	Урезает права до подключения, не после	Без вычеркнутой строки «все репозитории, включая приватные» воспроизводится документированная эксфильтрация (Invariant Labs, май 2025)
Субагент	Решаем — по сигналу вытеснения работы чтением	Делегирует задачу бесплатно, без единой строки конфигурации; чтение уходит в контекст субагента, обратно приходит итог	Рабочий каталог не разводит: исполнители правят там, где лежит работа, и столкновения убирают раскладкой зон

«Понедельник закрывает третья строка, среду — четвёртая. Четыре строки заполнены, ось занятия закрыта.»

Что появилось в репозитории

СХЕМА, НЕ СНИМОК

СТУПЕНЬ	ПУТЬ ИЛИ АРТЕФАКТ	СТАТУС
Хук	`.claude/settings.json` + `.claude/hooks/selftest-branch-guard.sh`	коммит есть
Скилл	`.claude/skills/deploy/SKILL.md`	по плану, коммита нет
CLAUDE.md	40 строк / 5 разделов	коммит есть
DECISIONS.md	документационный слой репозитория	без изменений
MCP	`.mcp.json`, права вычеркнуты до трёх операций	по плану, коммита нет и не будет
Субагент	`.claude/agents/diff-reviewer.md`, `tools: Read, Grep, Glob`	по плану, коммита нет и не будет
Рабочие копии	`CLAUDE.md`, раздел `Repository etiquette` — две строки: копия на задачу, слияние и уборка в конце	по плану, коммита нет и не будет

«Два файла этого занятия и строка правила останутся только планом. В публичном репозитории их не будет — и это сказано прямо, не спрятано.»

Этот вопрос уже звучал

> *«В начале занятия ответ на него вы держали при себе. Сейчас вслух: у кого он изменился и что именно его изменило.»*

«Работа вышла за контекст агента — что вы заводите первым: приносите внутрь то, чего внутри нет, или выносите наружу то, что внутрь не помещается?»

Чем занятие кончается

КЕЙС	ОПОРА	ПРОВЕРЯЕТСЯ ЧЕМ
MCP — данных нет в репозитории	есть	`claude mcp list` / `/mcp` и вызов, подписанный именем сервера, — у себя за минуту; сам собранный файл при этом целевой, коммита за ним нет
MCP — описания инструментов занимают контекст	нет	числа документации и инженерных блогов; замера на занятии не делаем
Субагент — чтение уходит в контекст исполнителя	нет	требует подготовленного параллельного прогона, не воспроизводится на занятии
Субагент — второй исполнитель, промежуток	нет	утверждение исследования (MAST); ни одна из двух опор занятия его не демонстрирует
Субагент — исполнители в одной папке	есть	`git worktree list` — у себя; и три столкновения в журнале репозитория, открытого на занятии

«Под частью сегодняшних карточек лежит команда, которую можно выполнить и получить ответ. Под другой частью — нет, и это названо прямо: необъявленный пробел стоил бы дороже объявленного.»

«Отсюда вы уносите один вопрос к своей работе: чего ей не хватает — того, чего внутри нет вовсе, или места, занятого чтением? Первое приносят внутрь, второе выносят наружу.»